IntelliKnow KMS

A Gen AI-powered Knowledge Management System — ask questions from Telegram or Microsoft Teams and get cited answers grounded in your organization's documents.

Telegram + Teams | RAG Knowledge Base | AI Intent Routing | AWS China · Docker Compose

1. Overview

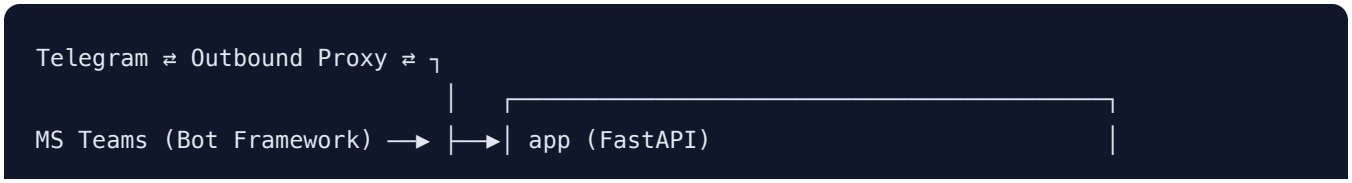
IntelliKnow KMS addresses fragmented enterprise information, inefficient knowledge retrieval, and siloed communication channels. It provides:

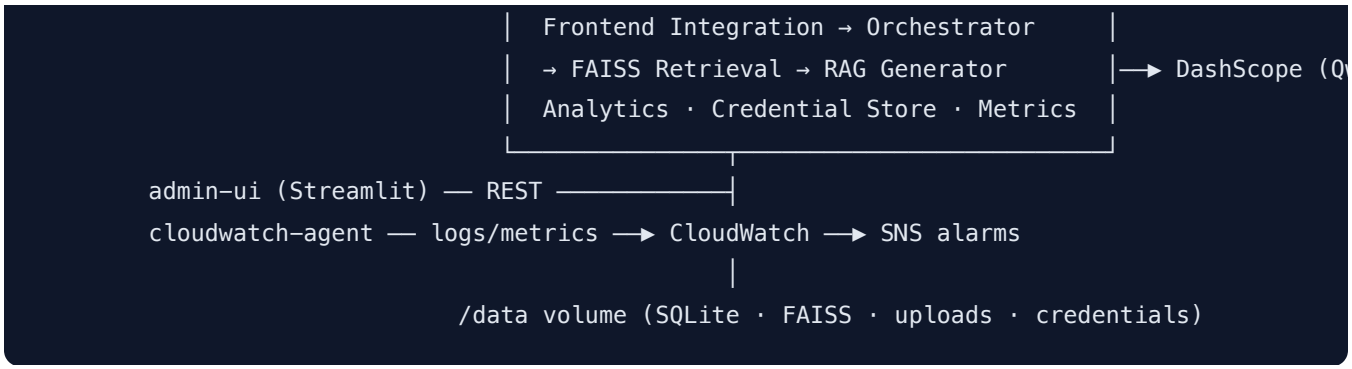
- **Multi-frontend integration** — end users query the knowledge base directly from Telegram and Microsoft Teams; responses are formatted natively per tool with a <3s p95 latency target.
- **Document-driven knowledge base** — upload PDF, DOCX, XLSX, TXT, or Markdown files (≤50 MB); content is AI-parsed (including embedded tables such as salary grids), chunked, embedded, and indexed for semantic search.
- **Intent orchestrator** — queries are classified by AI into intent spaces (HR, Legal, Finance, General, plus custom spaces) with a configurable confidence threshold (default 70%) and graceful fallback to General.
- **RAG responses with citations** — answers are generated exclusively from retrieved passages and always cite their source documents; a clear no-match message is returned when nothing relevant is found.
- **Admin web UI** — five screens (Dashboard, Frontend Integration, KB Management, Intent Configuration, Analytics) to operate the entire system.
- **Analytics & observability** — query history, classification accuracy, KB usage metrics, CSV export, plus CloudWatch metrics, logs, and alarms.

2. Tech Stack

Concern	Technology	Notes
Backend API & bots	Python 3.11, FastAPI, httpx (async)	Bot integration, orchestrator, RAG, admin REST API
Admin UI	Streamlit (multi-page app)	5 screens, card-based layout
Metadata & query log	SQLite (WAL mode)	Zero-ops persistence on a bind-mounted volume
Vector search	FAISS (IndexFlatIP, cosine)	One index per intent space; rebuilt from SQLite embeddings
LLM	Aliyun Tongyi Qwen-Max via DashScope API	Document structuring, intent classification, RAG generation
Embeddings	DashScope text-embedding-v3	Same vendor and credentials as the LLM
Document parsing	pypdf, pdfplumber, python-docx, openpyxl	Loader-per-format pattern; tables preserved as markdown
Deployment	Docker Compose on AWS China EC2	3 services: app, admin-ui, cloudwatch-agent
Monitoring	CloudWatch + SNS	Latency, error rate, integration health metrics and alarms
Testing	pytest, pytest-asyncio, Hypothesis, respx	Unit + property-based + integration tests

3. Architecture at a Glance





> **AWS China note:** Telegram is unreachable from AWS China regions, so the Telegram integration uses *long polling routed through an outbound proxy* (outbound-only — no inbound public endpoint needed). Teams and DashScope traffic goes direct.

Full details: see `docs/ARCHITECTURE.md`, `docs/HLD.md`, and `docs/LLD.md` (generated during implementation).

4. Prerequisites

- Docker and Docker Compose (v2) installed — on the EC2 host, enable Docker via systemd so containers restart on boot
- An AWS China EC2 instance (4 GB+ RAM recommended) with an IAM role or credentials permitting CloudWatch `PutMetricData`, CloudWatch Logs, and SNS publish
- An HTTPS forward proxy endpoint reachable from the instance with Telegram connectivity (for the outbound proxy)
- A DashScope API key (Aliyun Tongyi Qwen-Max)
- A Telegram bot token (from @BotFather) and a Microsoft Teams bot registration (App ID + password)

5. Setup

5.1 Clone and configure

```
# 1. Clone the repository
git clone <YOUR_REPO_URL> intelliknow-kms
```

```
cd intelliknow-kms

# 2. Create the host-only environment file (never committed to git)
sudo mkdir -p /opt/intelliknow
sudo tee /opt/intelliknow/.env > /dev/null <<'EOF'
CREDENTIAL_MASTER_KEY=<generate with: python -c "from cryptography.fernet import Fernet; print
TELEGRAM_PROXY_URL=http://<your-proxy-host>:<port>
AWS_REGION=cn-north-1
EOF
sudo chmod 600 /opt/intelliknow/.env

# 3. Create the persistent data directory
sudo mkdir -p /data/{uploads,faiss,credentials,logbuffer}
```

5.2 Build and start

```
docker compose --env-file /opt/intelliknow/.env up -d --build

# Admin UI:    http://<host>:8501
# Backend API: http://<host>:8000  (Teams webhook: POST /webhooks/teams)
```

5.3 Configure credentials (via Admin UI)

Open the Admin UI → **Frontend Integration** screen and enter:

- **DashScope**: API key (`sk-…`)
- **Telegram**: bot token
- **Teams**: App ID and App password

Credentials are validated before storage, saved to an encrypted store on the data volume (never in source code or git), masked in the UI (last 4 characters only), and redacted from all logs.

5.4 Provision monitoring

```
# Creates CloudWatch alarms (latency > 3s, error rate > 5%, integration unhealthy) wired to SN
python scripts/setup_cloudwatch_alarms.py --sns-topic <arn>
```

5.5 Verify

- Use the **Test** button on each integration card to run an end-to-end connectivity check
- Upload a sample document on **KB Management** and wait for status *Processed*

- Send a question to your bot in Telegram or Teams and confirm a cited answer arrives

6. Integration Guide

6.1 Telegram

1. Create a bot with @BotFather (`/newbot`) and copy the bot token.
2. Enter the token on the Admin UI → Frontend Integration → Telegram card and save.
3. The app starts a long-polling loop through the outbound proxy automatically — no webhook or public endpoint required.
4. Click **Test** to verify connectivity, then message your bot.

6.2 Microsoft Teams

1. Register a bot in the Bot Framework portal / Azure Bot Service; note the App ID and generate an App password.
2. Set the bot's messaging endpoint to `https://<your-host>/webhooks/teams` (HTTPS required — front the app with a TLS terminator or ALB).
3. Enter the App ID and password on the Admin UI → Frontend Integration → Teams card and save.
4. Add the bot to Teams (app manifest / sideload), click **Test**, then chat with it.

> **Latency SLO:** the pipeline targets <3 seconds end-to-end (p95) from message receipt to response delivery. A CloudWatch alarm fires if the p95 exceeds the configured threshold over 5 minutes.

7. Using the System

- **Dashboard** — integration status, document counts by status, 24-hour query activity.
- **KB Management** — drag-and-drop upload (PDF/DOCX/XLSX/TXT/MD, ≤50 MB), document table with search/filter, View/Delete/Update actions, intent space assignment.
- **Intent Configuration** — manage intent spaces (HR/Legal/Finance defaults + custom), define up to 50 keywords per space to guide classification, tune the confidence threshold

(0–100, default 70).

- **Analytics** — query history with filters, top-10 documents and intent spaces, per-space classification accuracy, admin verification of classifications, CSV export.

8. Testing

```
# Run the complete test suite (unit + property-based + integration) in one command
make test        # equivalent to: pytest
```

- **Unit tests** — document parsing, classification routing, per-tool response formatting, credential handling (nominal + error paths for each).
- **Property-based tests** — Hypothesis, one test per correctness property from the design (25 properties, ≥100 iterations each).
- **Integration tests** — end-to-end query flow with DashScope, Telegram, and Teams APIs mocked; FAISS and SQLite run real in temp directories.

9. Project Structure

```
intelliknow-kms/
├── app/                    # FastAPI backend
│   ├── bots/               # Telegram (long-poll via proxy) + Teams adapters
│   ├── core/               # Orchestrator (classify → route) + data models
│   ├── kb/                 # Loaders, processor pipeline, SQLite store, FAISS search
│   ├── ai/                 # DashScope client + prompt templates
│   ├── rag/                # RAG generator with citations
│   ├── security/           # Encrypted credential store + log redaction
│   ├── analytics/          # Query log, metrics, CSV export
│   ├── monitoring/         # CloudWatch publisher with local buffering
│   └── api/                # Admin REST endpoints
├── admin_ui/               # Streamlit app (5 screens)
├── tests/                  # unit / properties / integration
├── docs/                   # ARCHITECTURE.md · HLD.md · LLD.md · AI_USAGE_REFLECTION.md
├── scripts/                # CloudWatch alarm setup, demo seed
├── samples/                # Sample knowledge documents (incl. embedded-table PDF)
```

```
├── docker-compose.yml
└── Makefile
```

10. Operations Notes

- **Persistence** — SQLite, FAISS indexes, uploads, and credentials live under `/data` (host bind mount) and survive container restarts, recreation, and image updates.
- **Auto-recovery** — all services use `restart: unless-stopped`; containers come back within 5 minutes of an instance reboot and within 60 seconds of an unexpected exit.
- **Monitoring resilience** — if CloudWatch is unreachable, metrics and logs buffer locally under `/data/logbuffer` and resend on the next cycle; query processing is never interrupted.
- **Security** — no credentials in source code or version control; encrypted at rest; masked in the UI; redacted from logs, error messages, and stack traces.